EDUGENIE

Google Gemini Powered Learning Assistant

A Naan Mudhalvan Project Report

Project Domain: Generative Artificial Intelligence & Education Technology

Technology Stack: Python, FastAPI, HTML, CSS, Google Gemini API

Submitted By	Kaviya S
Programme	B.Tech Information Technology
College	Salem College of Engineering and Technology
Batch	2024 – 2028
Project	EduGenie: Google Gemini Powered Learning Assistant
Training	Naan Mudhalvan

Academic Project Report

CERTIFICATE

This is to certify that Kaviya S, a student of B.Tech Information Technology, Salem College of Engineering and Technology, has completed the project entitled “EduGenie: Google Gemini Powered Learning Assistant” as part of the Naan Mudhalvan project work.

The project demonstrates the use of Generative AI, Google Gemini API, FastAPI, Python and web technologies to create an interactive learning assistant for students.

Student Signature	Mentor Signature	HOD / Faculty
_____	_____	_____

ABSTRACT

EduGenie is a lightweight AI-powered educational assistant designed to make learning simpler, faster and more interactive. The system combines a FastAPI backend with a simple HTML and CSS interface and uses Google Gemini through an API for intelligent educational tasks. It allows learners to ask academic questions, simplify complex concepts, summarize long passages, generate multiple-choice quizzes and receive structured learning recommendations.

The project follows a modular architecture so that each educational function can be developed and tested independently. The application is intended to support school students, college students and self-learners by converting difficult or lengthy learning material into clear, accessible content. The project also demonstrates practical skills in API integration, prompt engineering, backend development, frontend integration and AI-assisted education.

Keywords: Generative AI, Google Gemini, FastAPI, Python, Educational Assistant, Quiz Generation, Summarization, Personalized Learning

1. INTRODUCTION

Artificial Intelligence is increasingly being used to support personalized and interactive learning. Traditional learning resources may contain complex explanations, lengthy text and large amounts of information that are difficult for beginners to understand quickly. EduGenie addresses this challenge by providing an easy-to-use AI learning assistant.

The system accepts a learner’s request through a web interface and routes it to the appropriate backend module. Depending on the selected task, EduGenie can answer a question, explain a concept in simple language, summarize text, generate a quiz or create a step-by-step learning path.

2. PROBLEM STATEMENT

Students often spend significant time searching for suitable explanations, revision material and practice questions. Generic learning resources may not match the learner’s current level. There is therefore a need for an accessible system that can provide concise explanations, instant answers, practice quizzes, summaries and structured learning guidance from a single interface.

3. OBJECTIVES

- Develop a simple AI-powered educational assistant.
- Provide concise answers to academic questions.
- Explain complex topics using beginner-friendly language.
- Generate MCQ quizzes from a topic or passage.
- Summarize lengthy educational content.
- Provide structured beginner-to-advanced learning paths.
- Demonstrate integration of Generative AI with a FastAPI web application.

4. EXISTING SYSTEM

In a conventional learning process, students use separate resources for explanations, search, quizzes, summaries and learning plans. Switching between different applications can consume time and may result in information that is too advanced or too broad for the learner’s needs.

Limitations: fragmented resources, limited personalization, time-consuming searching, and difficulty in converting long material into quick revision notes.

5. PROPOSED SYSTEM

EduGenie combines multiple learning-support functions in one lightweight web application. A learner selects a task, enters a topic or passage and submits the request. The FastAPI backend processes the request and calls the relevant AI module. The generated result is then displayed through the web interface.

Major features: Q&A, concept explanation, quiz generation, summarization and personalized learning recommendations.

6. SYSTEM ARCHITECTURE

The proposed architecture contains four major layers: User Interface, FastAPI Backend, AI Processing Modules and Google Gemini API. The frontend collects user input. FastAPI routes the request to the selected module. The module prepares an appropriate prompt and communicates with the AI service. The response is processed and returned to the frontend.

> Architecture Flow: User → HTML/CSS Interface → FastAPI REST API → Educational Module → Google Gemini API → Response Processing → Web Interface

7. HARDWARE AND SOFTWARE REQUIREMENTS

Category	Requirement
Operating System	Windows / macOS / Linux
Processor	Modern dual-core or better processor
RAM	4 GB minimum; 8 GB recommended
Python	Python 3.10 or later
Backend	FastAPI + Uvicorn
Frontend	HTML5 + CSS3
AI Service	Google Gemini API
Templating	Jinja2

8. PREREQUISITES AND INSTALLATION

Python: Install Python 3.10+ and verify using python --version. Ensure Python is added to PATH during installation.

FastAPI: Install the backend framework and supporting packages using pip.

Uvicorn: Install the ASGI server using pip install uvicorn.

Jinja2: Install the HTML templating engine using pip install jinja2.

Google Gemini API: Create an API key through Google AI Studio and store it securely. The key should not be hard-coded or publicly shared.

9. PROJECT FOLDER STRUCTURE

EduGenie/
├── main.py   # FastAPI application
├── explanation_module.py   # Concept explanation logic
├── qna.py   # Question answering
├── quiz_module.py   # Quiz generation
├── summary_module.py   # Summarization
├── learning_path.py   # Learning recommendations
├── templates/index.html   # Web interface
├── static/style.css   # Styling
└── requirements.txt   # Dependencies

10. MODULE DESIGN AND IMPLEMENTATION

10.1 Explanation Module

The explanation module converts difficult concepts into simple, readable descriptions. The prompt is designed to request clear language and an appropriate level of detail so beginners can understand the topic.

10.2 QnA Module

The QnA module sends the learner’s question to the Gemini service and returns an educational response. It can be used for general knowledge and academic questions.

10.3 Quiz Module

The quiz module generates three multiple-choice questions with four options each. The prompt requests structured output so the application can display questions and answers consistently. Response validation and JSON parsing are used to handle the generated data.

10.4 Summary Module

The summary module transforms long paragraphs into concise revision-friendly content while attempting to preserve the main ideas and context.

10.5 Learning Path Module

The learning-path module generates a structured progression from beginner to advanced topics. It can include suggested timelines and learning resources, helping users follow a systematic study plan.

11. FASTAPI REST ENDPOINTS

Endpoint	Purpose
/qa	Answer learner questions
/explain	Explain a concept simply
/quiz	Generate MCQ quiz
/summarize	Summarize educational text
/learn/recommendations	Generate learning path

12. FRONTEND DESIGN

The frontend is created using HTML and CSS. It contains a task selector, text input area, submit button and result container. The design is responsive so that the interface can be used on different

screen sizes.

The frontend sends a POST request to the appropriate FastAPI endpoint. Once the response is received, the result is displayed to the user without requiring a separate application.

13. PROJECT WORKFLOW

Step 1: User opens EduGenie.
Step 2: User selects Explain, QnA, Quiz, Summary or Recommend Path.
Step 3: User enters a question, topic or passage.
Step 4: Frontend sends a request to FastAPI.
Step 5: The relevant module prepares the AI prompt.
Step 6: Google Gemini generates the response.
Step 7: Backend validates/processes the response.
Step 8: Result is displayed on the webpage.

14. SAMPLE USE CASES

Use Case 1 – Q&A: A student asks, “Which is the largest ocean?” EduGenie provides a concise educational answer.

Use Case 2 – Quiz: A student enters “Pythagoras Theorem” and selects Generate Quiz. The system creates three MCQs with four options each.

Use Case 3 – Learning Path: A learner enters “SQL” and requests a learning path. EduGenie produces a staged plan from beginner concepts to advanced topics.

15. TESTING

Test Case	Input	Expected Result
QnA	Largest ocean?	Concise answer is displayed
Explain	Pythagoras Theorem	Simple explanation is displayed
Quiz	Photosynthesis	3 MCQs with 4 options
Summary	Long paragraph	Short summary is displayed
Learning Path	SQL	Beginner-to-advanced plan

16. DEPLOYMENT AND EXECUTION

After installing the required packages and configuring the Gemini API key, the FastAPI application can be started locally using:

```
uvicorn main:app --reload
```

The application can then be accessed through the local server address provided by Uvicorn, commonly http://127.0.0.1:8000.

17. SECURITY AND RESPONSIBLE AI CONSIDERATIONS

The Gemini API key must be kept private and should preferably be stored in an environment variable rather than directly inside source code. AI-generated responses should be reviewed for accuracy, especially when used for academic or factual content. Personal or sensitive information should not be unnecessarily submitted to the application.

18. ADVANTAGES

• Single interface for multiple learning tasks.
• Fast access to explanations and answers.
• Supports interactive quiz-based practice.
• Helps reduce long educational content into revision-friendly summaries.
• Provides structured learning recommendations.
• Modular design supports future enhancements.
• Lightweight web interface is easy to use.

19. LIMITATIONS

• AI-generated information can sometimes contain errors and should be verified.
• Gemini API access depends on network connectivity and API availability.
• API usage may be subject to quotas or service limits.
• Personalized recommendations depend on the quality of the user's input.
• The current version has limited persistent progress tracking.

20. FUTURE ENHANCEMENTS

Future versions can include voice-based interaction, multilingual support, mobile applications, student progress dashboards, gamification, adaptive learning paths, image/PDF-based doubt solving, teacher dashboards, group study features and integration with learning management systems such as Moodle or Google Classroom.

21. EXPECTED OUTCOME

The completed system is expected to provide students with an easy-to-use AI learning companion that supports question answering, concept clarification, practice, revision and structured learning. The project also provides practical exposure to Generative AI, API integration, backend development, frontend development and prompt engineering.

22. CONCLUSION

EduGenie demonstrates how Generative AI can be integrated into an educational web application to support learners through multiple functions. By combining Google Gemini, FastAPI, Python and a simple web interface, the system provides question answering, simplified explanations, quiz generation, summarization and learning recommendations in one platform.

The modular architecture makes the application easy to test and extend. The project also highlights important practical areas such as API integration, prompt design, response processing, user interface development and responsible use of AI-generated information. With additional features such as multilingual support, voice interaction, progress tracking and adaptive learning, EduGenie can be extended into a more comprehensive digital learning companion.

23. REFERENCES

1. Google AI for Developers – Gemini API documentation.
2. FastAPI Documentation – Web framework for building APIs with Python.
3. Python Documentation – Python programming language reference.
4. Uvicorn Documentation – ASGI server for Python applications.

5. Jinja2 Documentation – Templating engine for Python.
6. HTML and CSS documentation – Web interface development references.

PROJECT SUBMISSION DETAILS

Student Name	Kaviya S
Course	B.Tech Information Technology
College	Salem College of Engineering and Technology
Project Title	EduGenie: Google Gemini Powered Learning Assistant
Programme	Naan Mudhalvan
Academic Year	2026